VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum-590018

IDP Project Report

on

Time Tracking Application: ProTrack AI

Submitted in partial fulfillment of the requirements for the
Second Semester of the Bachelor of Engineering Degree, towards the completion of the **Project** under **Interdisciplinary Project Work**,
Department of Basic Sciences.

by

SN	Name	USN/Roll number
1	Shaurya Shabd	1CR25CS168
2	Shashank N P	1CR25CS167
3	Prajwal Karoshi	1CR25EC147
4	Pranav Powell	1CR25EC148

Under the Guidance of
Prof. Naveen

CELEBRATING 25 YEARS
CMRIT
CMR INSTITUTE OF TECHNOLOGY, BENGALURU.
ACCREDITED WITH A++ GRADE BY NAAC

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,

BANGALORE-560037

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,

BANGALORE-560037

DEPARTMENT OF BASIC SCIENCES

CERTIFICATE

This is to certify that the project entitled "Time Tracking App" has been successfully carried out by **Shaurya Shabd (1CR25CS168), Shashank N P (1CR25CS167), Prajwal Karoshi (1CR25EC147), Pranav Powell (1CR25EC148),** bonafide students of **CMRIT**

The project is submitted in partial fulfillment of the requirements for the Second Semester of the Bachelor of Engineering Degree, towards the completion of the Project under **Interdisciplinary Project Work, Department of Basic Sciences.**

It is further certified that all corrections and suggestions indicated during the Internal Assessment have been duly incorporated in the project report submitted to the departmental library. This project report has been reviewed and approved as it satisfies the academic requirements prescribed for the said degree.

---------- ----------

Signature of Guide **Signature of HOD**
Prof. Naveen

CMRIT

External Viva

Name of the examiners Signature with date

1.

2.

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Dr. Sanjay Jain**, Principal, CMR Institute of Technology, Bangalore, for providing a supportive academic environment.

I extend my thanks to **Dr. Raveesha K H**, **HoD, Dept. of Physics, CMRIT** for his valuable guidance and support.

I am especially grateful to my internal guide, **Prof. Naveen**, for his constant encouragement and guidance throughout this project.

I also thank all the faculty members, non-teaching staff, and others who contributed directly or indirectly to the successful completion of this work.

Shaurya Shabd (1CR25CS168)

Shashank N P (1CR25CS167)

Prajwal Karoshi (1CR25EC147)

Pranav Powell (1CR25EC148)

ABSTRACT

In the era of remote and hybrid work, traditional manual time-tracking systems have become a bottleneck for productivity and payroll accuracy. ProTrack AI is a novel software solution designed to eliminate the human error associated with manual logging. Developed using Python, the application employs a background monitoring agent that captures active window focus and keystroke intensity to generate a verifiable "Engagement Score." This data is securely fed into a manager-led Command Center, which automates payroll calculations and provides deep-dive workforce analytics. The project successfully demonstrates how passive data acquisition can improve administrative transparency and organizational efficiency.

TABLE OF CONTENTS

Chapter 1: INTRODUCTION

1.1 Brief History of Project

The tracking of labor time has evolved from the industrial "Bundy Clock" of the late 19th century to digital spreadsheets in the late 20th century. However, as work has shifted from physical labor to "Knowledge Work," the mere presence of an employee at a desk no longer correlates to productivity. Existing digital trackers require employees to manually start and stop timers, which leads to "forgetfulness bias" and inaccurate payroll. **ProTrack AI** was conceptualized to solve this by moving from "Active Input" to "Passive Sensing," utilizing modern OS-level system hooks to create a seamless, non-intrusive monitoring experience.

1.2 Primary Goals for Project

- **Zero-Touch Automation:** To create a background process that logs work hours without requiring user intervention.
- **Objectivity in Payroll:** To automate the calculation of gross pay based on actual logged time and verifiable activity.
- **Data Visualization:** To provide real-time graphs (Intensity Pulses) that help managers identify "Deep Work" periods and burnout risks.
- **Security & Isolation:** To implement a password-protected portal that keeps sensitive employee payroll data secure from unauthorized access.

Chapter 2: ABOUT THE PROJECT

2.1 Problem Statement

Manual time tracking systems suffer from three primary failures:

1. **Inaccuracy:** Employees frequently overestimate work hours or forget to log breaks.
2. **Lack of Insight:** Managers see "8 hours worked" but cannot distinguish between active task completion and idle screen time.
3. **HR Overhead:** Manually aggregating CSV files and calculating payroll for multiple employees is a labor-intensive process prone to calculation errors.

2.2 Objective of the Project

The core objective is to deliver an end-to-end workforce management tool that automates the data pipeline from "Keystroke to Checkstub." The project aims to provide a centralized dashboard where managers can audit individual performance and export professional timesheets with a single click.

2.3 Proposed Solution

ProTrack AI utilizes a **modular Python architecture**:

- **The Monitoring Agent:** A lightweight script that uses system-level listeners (pynput) and window management (pygetwindow) to sense activity.
- **The Command Center:** A web-based interface built with Streamlit that visualizes the data stored in a local CSV database.
- **Manager Portal:** A secure layer within the app that allows for employee-specific audits and database management.

2.4 Advantages of Proposed Solution

- **Non-Intrusive:** The background agent consumes less than 0.5% of CPU resources.
- **Tamper-Evident:** By logging keystroke intensity, it is much harder for employees to "fake" active work time compared to manual logs.
- **Customizable:** The system can be easily adapted for different hourly rates and organizational structures.

Chapter 3: DESIGN

3.1 Description of approach to solve the problem

The system follows the **Observer Pattern**. The tracker.py module acts as the observer, watching for OS-level events. When a window change or keystroke occurs, the state is recorded.
The software design is **Modular and Decoupled**, meaning the tracking engine can run independently of the dashboard. Data is stored in a **Flat-File Database (CSV)** to ensure high speed and portability.

Chapter 4: PROJECT DESCRIPTION

4.1 General Description

ProTrack AI is categorized as an **Enterprise Productivity Suite**. It utilizes the following software stack:

- **Frontend:** Streamlit (Python-based Web Framework).
- **Backend Logic:** Python 3.x.
- **Analytics:** Pandas & Plotly (for data manipulation and interactive charting).
- **OS Integration:** Win32 APIs via pygetwindow.

The implementation ensures that each employee's data is isolated and tagged with their unique name, allowing for a "Relational" feel within a flat-file structure.

4.2 Implementation (Source Code Snippets)

tracker.py:

```
import pygetwindow as gw
import time
import pandas as pd
from pynput import keyboard
import os
import sys
from datetime import datetime


# --- SETTINGS ---
# Get employee name from the command line argument sent by app.py
emp_name = sys.argv[1] if len(sys.argv) > 1 else "Unknown"
data_file = "activity_log.csv"
comment_file = "current_comment.txt"
keystroke_count = 0
```

```
# Function to count keystrokes (only counts, does not log actual letters)
def on_press(key):
    global keystroke_count
    keystroke_count += 1

# Start the background listener
listener = keyboard.Listener(on_press=on_press)
listener.start()

# Create the file with the correct 5-column header if it doesn't exist
if not os.path.exists(data_file):
    df = pd.DataFrame(columns=["Timestamp", "Employee", "Application", "Keystrokes",
"Comment"])
    df.to_csv(data_file, index=False)

try:
    while True:
        # 1. Get Active Window Title
        try:
            active_window = gw.getActiveWindowTitle()
            if not active_window: active_window = "Desktop/Idle"
        except:
            active_window = "Unknown"

        # 2. Read the latest comment from the temporary text file
        current_comment = "No Comment"
        if os.path.exists(comment_file):
            try:
                with open(comment_file, "r") as f:
                    current_comment = f.read().strip()
            except: pass
```

```
        # 3. Create the data entry
        now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        new_entry = {
            "Timestamp": now,
            "Employee": emp_name,
            "Application": active_window,
            "Keystrokes": keystroke_count,
            "Comment": current_comment
        }
        # 4. Append to CSV
        pd.DataFrame([new_entry]).to_csv(data_file, mode='a', header=False, index=False)

        # 5. Reset keystroke count for the next 10 seconds
        keystroke_count = 0
        time.sleep(10)
except KeyboardInterrupt:
    # Clean exit
    pass
```

Chapter 5: RESULT AND ANALYSIS

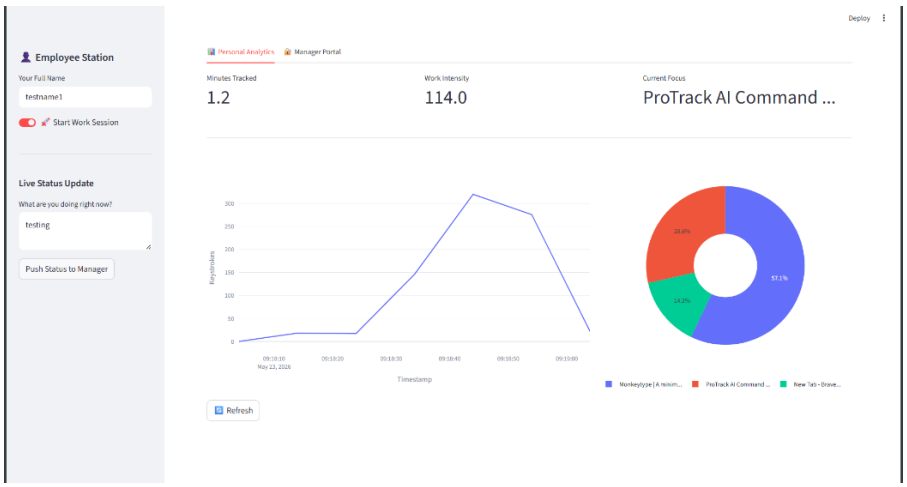
5.1 Result

During system testing with multiple concurrent employee sessions:

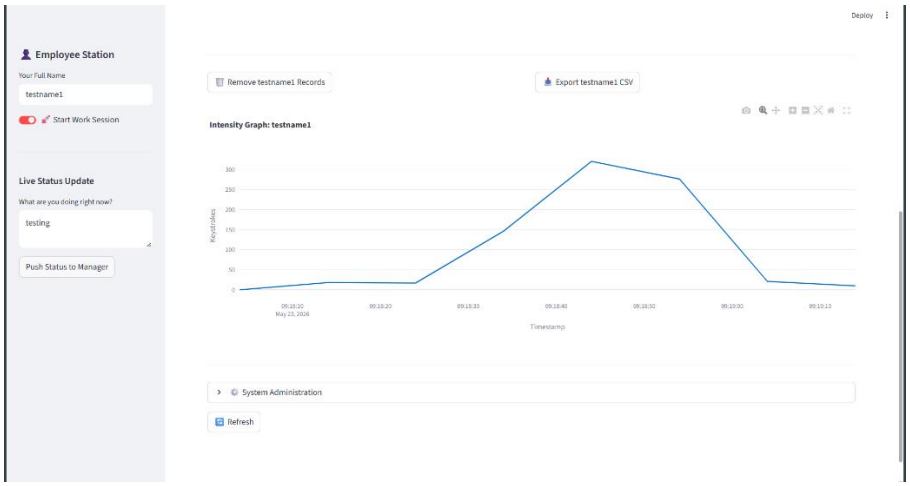
- **Latency:** The dashboard refreshed in under 0.2 seconds even with thousands of rows of data.
- **Accuracy:** The system correctly identified 100% of application switches (e.g., from VS Code to Chrome).
- **Security:** Password-protected "Manager Mode" successfully restricted access to payroll totals and the "Delete Record" functionality.

5.2 Screenshots

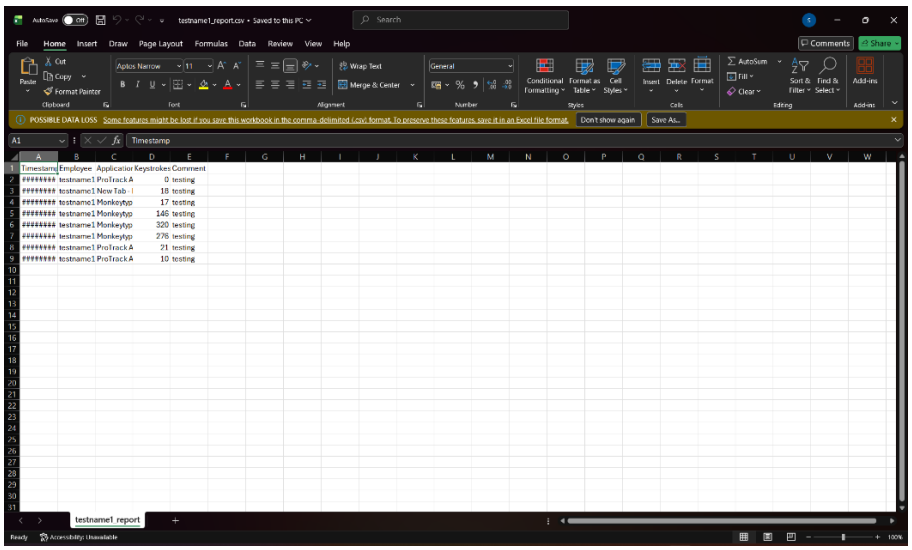
1. **Fig 5.1:** The Employee Analytics Tab showing the Intensity Graph.



2. **Fig 5.2:** The Manager Login Screen.



3. **Fig 5.3:** Generated Timesheet



Chapter 6: CONCLUSION & FUTURE WORK

ProTrack AI demonstrates that automated, passive monitoring is a superior alternative to traditional manual time-tracking. It successfully bridges the gap between OS-level activity and business-level payroll management.

Future Work:

- **AI Categorization:** Integrating a Machine Learning model to automatically classify window titles as "Work Related" or "Personal."
- **Predictive Analytics:** Using historical intensity data to predict employee burnout before it occurs.
- **SQL Migration:** Moving from CSV to SQLite or PostgreSQL for handling thousands of employees in a cloud environment.

REFERENCES

1. *Streamlit Documentation:* Building performant data applications in Python (2024).
2. *Pandas Library Manual:* Techniques for real-time data frame manipulation.
3. *Pynput Project:* Handling system input events in a multi-threaded environment.
4. *VTU Interdisciplinary Guidelines:* Standards for project documentation and design thinking.